SOFTWARE REQUIREMENTS SPECIFICATION

UniOne

Student Collaboration, Community Networking and Career Discovery Platform

Group Members

#	Name	Roll No.
1	Ali Haroon	SU92-BSSEM-F24-296
2	Sohaib Ammar	SU92-BSSEM-F24-339
3	Raheem Rizwan	SU92-BSSEM-F24-303
4	Ameer Ali	SU92-BSSEM-F24-310
5	M. Farhan Khan	SU92-BSSEM-F24-281
6	Faizan Asghar	SU92-BSSEM-F24-279
7	Muhammad Hammad	SU92-BSSEM-F24-284

Project	UniOne (Flutter mobile app + NestJS backend + React admin dashboard)
Institution	Superior University
Program	BS Software Engineering
Document	Software Requirements Specification (IEEE 830 / ISO 29148 style)
Version / Date	1.0 / 30 October 2026

Revision History

Version	Date	Description	Author
0.1	15 Oct 2026	Initial draft from research and architecture reports	UniOne Group
1.0	30 Oct 2026	Complete SRS with requirements and UML models	UniOne Group

Table of Contents

1. Introduction **5**

1.1 Purpose . . . 5

1.2 Project Introduction . . . 5

1.3 Scope . . . 5

1.4 Definitions, Acronyms and Abbreviations . . . 6

1.5 References . . . 6

1.6 Document Overview . . . 6

2. Overall Description **7**

2.1 Product Perspective . . . 7

2.2 Product Functions (Summary) . . . 7

2.3 User Classes and Characteristics . . . 8

2.4 Operating Environment . . . 8

2.5 Design and Implementation Constraints (summary) . . . 8

2.6 Assumptions and Dependencies . . . 8

3. Functional Requirements **9**

3.1 Authentication and Verification . . . 9

3.2 Onboarding and Profile . . . 9

3.3 Communities . . . 9

3.4 Discussions and Feed . . . 9

3.5 Collaboration Requests and Project Workspaces . . . 10

3.6 Real-Time Messaging . . . 10

3.7 Opportunities . . . 10

3.8 Notifications and Search . . . 10

3.9 Moderation, Safety and Appeals . . . 11

3.10 Administration . . . 11

3.11 Role Permission Matrix . . . 11

4. External Interface Requirements **12**

4.1 User Interfaces . . . 12

4.2 Hardware Interfaces . . . 12

4.3 Software Interfaces . . . 12

4.4 Communication Interfaces . . . 12

5. Non-Functional Requirements **13**

6. Constraints, Assumptions and Dependencies **14**

6.1 Constraints . . . 14

6.2 Risks and Mitigations . . . 14

7. Business Rules **14**

8. Use Case Specifications **15**

9. System Models (UML Diagrams and Workflows) **17**

9.1 Use Case Diagram 17

9.2 Data Flow Diagram (Level 0) 18

9.3 Class Diagram 19

9.4 Entity Relationship Diagram 20

9.5 Sequence Diagrams 21

9.6 Activity Diagrams (Workflows) 23

9.7 State Machine Diagrams 26

9.8 Deployment Diagram 26

10. Delivery Plan and Appendices **27**

10.1 MVP Backlog Priorities 27

10.2 Thin Vertical Slice and 14-Day Plan 27

10.3 Pre-Scale Validation Gates 27

10.4 Requirements Traceability (User Story to Requirements) 28

10.5 Technology Stack Summary 28

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements, constraints, and design models for **UniOne**, a mobile-first platform for student collaboration, community networking and career discovery at Superior University. It is the agreed reference for developers, testers, supervisors and stakeholders, and it is derived from the project's technology-stack research and master blueprint.

1.2 Project Introduction

Students today split their academic and professional lives across WhatsApp and Discord groups (unstructured and short-lived), Facebook Groups (weak moderation, no professional identity) and LinkedIn (costly job posts, no peer-collaboration tools). UniOne unifies these in a single ecosystem built on **verified university identity**. Its core ideas are:

- **Identity drives permissions:** rights such as posting internships are tied to verified roles.
- **Low friction for discovery, strict control for risk:** anyone verified can browse; high-stakes actions are reviewed.
- **Durable knowledge over fleeting chat:** threaded discussions with New/Best sorting; real-time chat is reserved for project teams and private groups.
- **Structured opportunities:** job, internship and collaboration posts are schema-bound objects that auto-expire (14 days).

1.3 Scope

In scope (MVP):

- University-email verification, onboarding and academic profiles.
- Seven community types: Open Interest, University, Academic/Study, Project, Career/Internship, Official Announcements, Private Collaboration Group.
- Threaded discussions, voting, New/Best sorting, search and notifications.
- Structured collaboration requests, project workspaces with real-time team chat.
- Verified-organization internship/job listings with pre-publication review and applications.
- Direct messages with a Message Request queue, block/report tools.
- Reason-coded moderation, appeals, immutable audit logs, and a web admin dashboard.
- Android and iOS app built in Flutter; NestJS backend; PostgreSQL; Socket.IO real-time.

Out of scope (deferred to Phase 2/3): Industry Network, Employer Talent Channels and Mentorship communities, voice/video (LiveKit), AI matching and semantic search, end-to-end encryption, microservices/Kafka/Kubernetes, multi-region deployment.

1.4 Definitions, Acronyms and Abbreviations

Term	Meaning
SRS	Software Requirements Specification
MVP	Minimum Viable Product (first release)
Community	A topic or institutional container for posts, members and rules
Verified Org	Employer account whose corporate domain has been verified
Message Request	Inbox queue for first-contact messages from non-connections
Outbox	Local (client) or database (server) table of pending events written before sending, used for reliable delivery
Idempotency key	Client-generated UUID that makes retries safe (no duplicate messages)
Drift	Reactive SQLite library used as the app's local source of truth
RBAC / IDOR	Role-based access control / Insecure direct object reference
FTS	PostgreSQL full-text search
OTP	One-time password sent to the university inbox

1.5 References

- UniOne - Flutter-First Technology Stack and System Architecture Report (Oct 2026).
- UniOne Platform - Consolidated Master Blueprint and Technical Architecture (Oct 2026).
- IEEE Std 830-1998 and ISO/IEC/IEEE 29148:2018 - Requirements engineering.
- OMG Unified Modeling Language (UML) 2.5; OpenAPI Specification 3.1; OWASP Mobile Top 10 (2024) and API Security Top 10 (2023).

1.6 Document Overview

Section 2 describes the product in context; Section 3 lists functional requirements; Section 4 external interfaces; Section 5 non-functional requirements; Section 6 constraints, assumptions and dependencies; Section 7 business rules; Section 8 use-case specifications; Section 9 contains all UML models and workflows; Section 10 the delivery plan and appendices.

2. Overall Description

2.1 Product Perspective

UniOne is a new, self-contained system made of three deployable parts from one monorepo: a Flutter mobile client, a NestJS modular monolith (REST API, Socket.IO gateway and BullMQ workers) and a React admin dashboard. Data lives in PostgreSQL (source of truth), Valkey/Redis (presence, pub/sub, queues) and Cloudflare R2 (media). The mobile app treats its local Drift database as the single source of truth for the UI; REST and WebSocket events reconcile into it, which gives offline support and reliable chat.

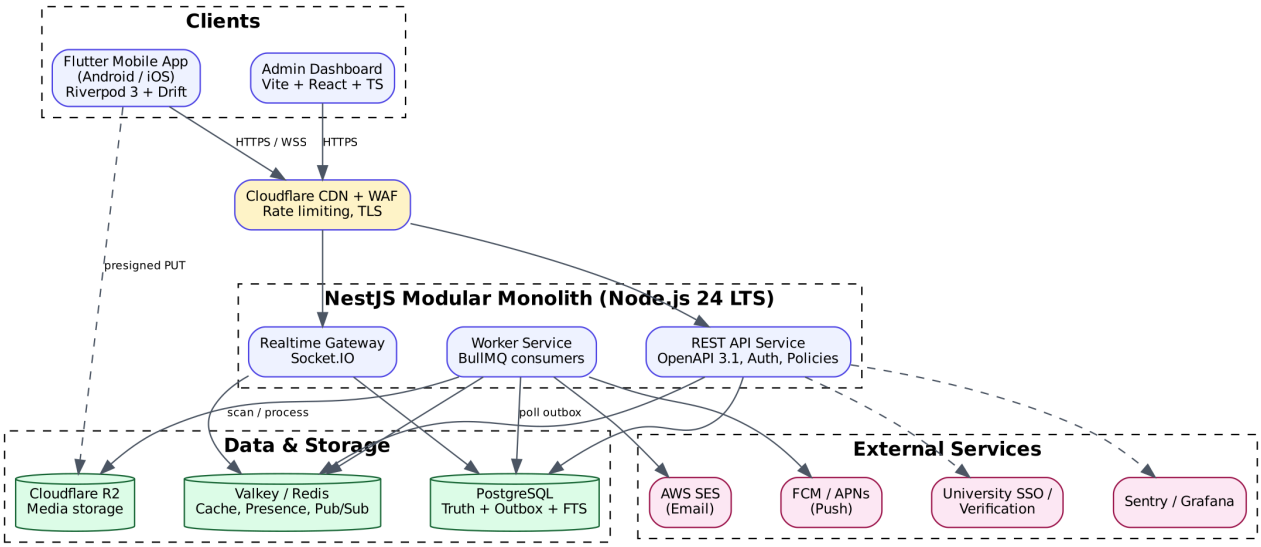


Figure 2.1 - High-level system architecture

2.2 Product Functions (Summary)

- Authentication, university verification and onboarding
- Profiles and privacy controls
- Communities (7 types), join gates and roles
- Discussion feed with threaded comments and voting
- Collaboration requests and private project workspaces
- Real-time chat and message requests
- Verified opportunity listings and applications
- Notifications (in-app and push)
- Search (people, communities, posts)
- Reporting, moderation, appeals and audit logs
- Administration dashboard

2.3 User Classes and Characteristics

User class	Description	Technical skill
Student / Member	Verified university student or alumnus; primary user	Low-Medium
Community Owner	Creates and configures a community, appoints moderators	Low-Medium
Community Moderator	Handles reports in assigned communities	Medium
Verified Organization	Employer or university office posting listings and announcements	Medium
Platform Admin	Manages users, organizations, policies, platform-wide safety and audit	High
System / Services	Email, push (FCM/APNs), SSO, storage - external actors	N/A

2.4 Operating Environment

- Mobile: Android (API 24+) and iOS (15+), built with Flutter (current stable).
- Admin dashboard: current Chrome, Edge, Firefox, Safari (desktop).
- Server: Node.js 24 LTS containers (Railway or ECS Fargate), managed PostgreSQL, Valkey/Redis, Cloudflare R2/CDN.

2.5 Design and Implementation Constraints (summary)

Flutter/Dart is a fixed decision for mobile. Backend uses TypeScript/NestJS. OpenAPI 3.1 is the contract. Student-budget hosting (about USD 35-60/month at pilot) restricts the design to managed services and a monolith. See Section 6 for the full list.

2.6 Assumptions and Dependencies

- Students hold active university email addresses on a known domain (e.g. @superior.edu.pk).
- Third-party services (FCM, APNs, SES, Cloudflare, managed Postgres) remain available at the assumed pricing.
- University leadership will nominate announcers and career-office accounts.
- Planning load: 100,000 registered users, about 20% daily active, about 1,000 peak concurrent connections.

3. Functional Requirements

Priority: **P0** = launch-critical, **P1** = high, **P2** = later phase. Each requirement is uniquely identified and testable.

3.1 Authentication and Verification

ID	Requirement	Pri.
FR-01	The system shall allow registration with a university email and password (argon2id hashing).	P0
FR-02	The system shall validate the email domain against an allowed list and send an OTP / magic link to the inbox.	P0
FR-03	The system shall mark the user VERIFIED (1-year expiry) after successful token confirmation and allow retry and resend.	P0
FR-04	The system shall auto-enroll verified students into their institutional community and block private university content before verification.	P0
FR-05	The system shall issue short-lived access tokens and rotating refresh tokens; users can view devices and revoke sessions.	P0
FR-06	The system shall support password reset with generic responses that prevent account enumeration.	P0

3.2 Onboarding and Profile

ID	Requirement	Pri.
FR-07	The system shall let the user choose an academic discipline and up to 3 interest chips, editable later.	P0
FR-08	The system shall generate a starter feed of 3 matching communities and 1 campus announcement (no empty feed).	P1
FR-09	The system shall provide a profile (bio, skills, projects) with field-level visibility controls and a verified badge.	P0

3.3 Communities

ID	Requirement	Pri.
FR-10	The system shall support seven MVP community types with their visibility, join model, creation and posting rights.	P0
FR-11	The system shall show a preview card (description, rules, member count, recent activity) before joining.	P0
FR-12	The join button shall adapt to the community: Instant Join, Request Access (with up to 3 screening questions) or Invite-Only.	P0
FR-13	Owners shall configure rules, join policy, and appoint or remove moderators.	P0
FR-14	Official Announcements communities shall be read-only except for designated announcers.	P0

3.4 Discussions and Feed

ID	Requirement	Pri.
FR-15	Members shall create, edit and delete posts (discussion, collaboration request, resource) with code blocks and images.	P0
FR-16	The system shall support threaded comments and voting, sortable by New and Best.	P0
FR-17	The feed shall use keyset (cursor) pagination and render from the local Drift database, including offline.	P0
FR-18	Every post and comment shall expose Report and Block actions.	P0

3.5 Collaboration Requests and Project Workspaces

ID	Requirement	Pri.
FR-19	Students shall publish a collaboration request with title, required skills, hours per week, capacity and expiration date.	P1
FR-20	Expired requests shall auto-close and leave an audit trace.	P1
FR-21	The system shall create a private project workspace limited to the approved roster, with a milestones page and a team chat channel.	P1
FR-22	Project updates shall not appear in the public feed.	P1

3.6 Real-Time Messaging

ID	Requirement	Pri.
FR-23	The system shall deliver messages in real time over Socket.IO with persist-before-ack semantics.	P0
FR-24	The client shall send messages through a local outbox with an idempotency key and show pending, sent, delivered, read or failed states.	P0
FR-25	After reconnect the client shall send lastSeq per conversation and the server shall return missed messages (no loss, no duplicates).	P0
FR-26	First-contact messages from non-connections shall land in a Message Request queue with media hidden; the recipient can Accept, Ignore, Block or Report.	P0
FR-27	The system shall support typing indicators, presence and coalesced read receipts.	P1
FR-28	Recruiter cold direct messages shall be blocked.	P0

3.7 Opportunities

ID	Requirement	Pri.
FR-29	Only verified organizations (or the Career Office) shall create internship/job listings with deadline, compensation, location and screening questions.	P0
FR-30	Listings shall pass a pre-publication review queue and expire automatically after 14 days.	P0
FR-31	Students shall apply with their UniOne profile and answers; organizations see applicants and status.	P0

3.8 Notifications and Search

ID	Requirement	Pri.
FR-32	The system shall send in-app notifications and push (FCM/APNs) with generic payloads (no message text), respecting user preferences and quiet hours.	P0
FR-33	The system shall provide search over people, communities and posts using PostgreSQL FTS and trigram matching, filtered by university, department and visibility, and excluding blocked users.	P1

3.9 Moderation, Safety and Appeals

ID	Requirement	Pri.
FR-34	Users shall report content or accounts with a category (e.g. harassment, fake internship).	P0
FR-35	Moderators shall remove content or sanction users only with a mandatory rule-derived reason code.	P0
FR-36	Every moderation action shall be written to an append-only audit log (actor, target, reason, prior state, timestamp, IP).	P0
FR-37	The system shall notify the affected author with the rule citation and an Appeal link; appeals go to a different reviewer.	P0
FR-38	Severe site-wide violations (scams, impersonation) shall escalate to the Platform Safety queue, with account suspension and session invalidation.	P0
FR-39	Moderators cannot review their own decisions; community owners cannot edit or delete audit logs.	P0

3.10 Administration

ID	Requirement	Pri.
FR-40	Admins shall manage users, organizations, communities and policies through a web dashboard with mandatory MFA.	P0
FR-41	The dashboard shall provide moderation queues with server-side pagination, filters and bulk actions authorized per record.	P0
FR-42	Admins shall view the full platform audit log and basic analytics (users, posts, reports, verification rate).	P1
FR-43	The system shall provide data export and account deletion that propagates to search, media and queues.	P1

3.11 Role Permission Matrix

Capability	Student	Owner	Moderator	Admin	Verified Org
View public community	Yes	Yes	Yes	Yes	Yes
Join private community	Request/Invite	Sets policy	Approves applicants	Override	Per rules
Publish discussion	If member	Yes	Yes	Yes (notice)	If member
Publish job/internship	No	No	No	Career team only	Yes (verified)
Remove community post	Own only	Yes	Yes	Site-wide	Own only
Appoint moderators	No	Yes	No	On policy breach	No
Issue sanction	No	Yes (reason)	Yes (reason)	Platform ban	No
Appeal decision	Yes	Not own	Not own	Final arbitration	If affected
Inspect audit logs	Own notices	Community log	Report queue	Full log	Own listings

4. External Interface Requirements

4.1 User Interfaces

- Mobile app: four root tabs (Home feed, Search, Chat, Profile) using Material 3, dark mode and text scaling; accessible labels for screen readers.
- Key screens: onboarding with starter feed, community preview and join, post composer (Discussion / Collab Request / Resource), opportunity card with expiry, chat and message requests, moderation notices with Appeal button.
- Admin dashboard: data-dense tables, moderation queue, user/organization management, audit log; keyboard accessible.

4.2 Hardware Interfaces

Device camera and gallery for image upload (EXIF GPS stripped), and network connectivity. No special hardware is required.

4.3 Software Interfaces

Interface	Purpose	Protocol
UniOne REST API (NestJS)	All CRUD operations; contract defined in OpenAPI 3.1	HTTPS / JSON
Socket.IO gateway	Real-time chat, presence, events (versioned envelope)	WSS
PostgreSQL	System of record, FTS, outbox	TCP / TLS
Valkey / Redis	Pub/sub, presence TTL keys, BullMQ queues	TCP / TLS
Cloudflare R2 + CDN	Presigned direct uploads, signed media URLs	HTTPS (S3 API)
FCM / APNs	Push notifications	HTTPS
AWS SES	Transactional email (OTP)	HTTPS / SMTP
University SSO (later)	SAML/OIDC institutional verification	SAML / OIDC
Sentry / Grafana	Error tracking and metrics	HTTPS

4.4 Communication Interfaces

- All traffic over TLS 1.2+. REST responses use a common error envelope and cursor pagination.
- WebSocket events use the envelope {type, version, eventId, occurredAt, data}; changes are additive-only.
- Mobile API compatibility policy: current plus two previous app versions (N-2).

5. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	p95 REST latency below 300 ms for feed and conversation list at the planning peak (about 200 RPS).
NFR-02	Performance	Message end-to-end delivery to an online recipient within 1 s (p95); app cold start below 2.5 s on a mid-range Android device.
NFR-03	Performance	Flutter p95 frame time below 20 ms on a low-end Android reference device; windowed lists for long chats.
NFR-04	Scalability	Support 100,000 registered users (about 20,000 DAU, 1,000 concurrent) on the base topology; scale via named triggers (replicas, PgBouncer, partitioning).
NFR-05	Reliability	Zero message loss: persist-before-ack, at-least-once fan-out, client de-duplication; transactional outbox for notifications.
NFR-06	Availability	99.5% monthly availability target at MVP; rolling deployments with at least 2 replicas of realtime/api at growth.
NFR-07	Offline	Previously loaded feeds, conversations and profiles are readable offline; sends queue and retry automatically.
NFR-08	Security	Deny-by-default server-side authorization on every object (anti-IDOR); automated cross-user access tests in CI.
NFR-09	Security	argon2id passwords, rotating refresh tokens in secure storage, session revocation that also closes live sockets, rate limits per IP + account + device.
NFR-10	Security	Uploads quarantined, magic-byte verified, malware-scanned, EXIF stripped before publication; signed URLs for private media.
NFR-11	Privacy	TLS everywhere, encryption at rest, generic push payloads, data export/delete, documented retention. The system does not claim end-to-end encryption.
NFR-12	Usability	Onboarding under 3 minutes; verification completion at least 70%; accessible labels and text scaling; core actions within 3 taps.
NFR-13	Maintainability	Modular monolith with lint-enforced module boundaries; OpenAPI-first generated clients; unit, integration and contract tests; at least 70% coverage on core modules.
NFR-14	Compatibility	Backward compatibility for N-2 app versions; expand-migrate-contract schema changes.
NFR-15	Auditability	Moderation, admin, session and consent actions stored in append-only logs.
NFR-16	Recoverability	Daily PITR backups plus weekly logical dump; restore drill before launch (RTO 1 h, RPO 5 min).
NFR-17	Cost	Infrastructure about USD 35-60/month (1K users), 65-120 (10K), 205-380 (100K).
NFR-18	Observability	Structured logs with request IDs, metrics and alerts for latency, queue lag and error rate; PII redaction.

6. Constraints, Assumptions and Dependencies

6.1 Constraints

ID	Constraint
C-01	Mobile client must be implemented in Flutter/Dart (Android + iOS) with Riverpod 3, go_router, Dio, Drift.
C-02	Backend must be a TypeScript NestJS modular monolith with separate api, realtime and worker deployables from one image.
C-03	PostgreSQL is the only system of record; no dual-writes to other stores (use the outbox).
C-04	No microservices, Kafka, Kubernetes, Elasticsearch or multi-region at MVP.
C-05	Admin dashboard must be a web SPA (Vite + React + TypeScript) behind SSO/VPN; Flutter Web is not used for it.
C-06	Budget restricts hosting to managed or low-cost container platforms.
C-07	Academic timeline: MVP vertical slice within 14 days of implementation; full MVP within the semester.
C-08	Legal: FERPA/GDPR/COPPA-style obligations depend on jurisdiction; an age policy and counsel review are required before public launch.
C-09	Apple Developer Program (USD 99/year) and Google Play account are required for store release.
C-10	OpenAPI 3.1 is the single source of truth for the REST contract; generated code is never hand-edited.

6.2 Risks and Mitigations

Risk	Mitigation
Message loss or duplication on reconnect	Persist-first, idempotency keys, per-conversation seq, cursor replay, chaos tests
Contract drift between Dart, TypeScript and backend	OpenAPI-first, CI breaking-change diff, shared fixtures
Low-end Android jank	Device-lab performance budgets from the start
Private data exposure	Central policy engine, deny-by-default, CI cross-user tests
Low verification or employer participation	Pilot with 15-20 students and 5-8 employers; adjust gates
Moderator burnout / unsafe content	Reason codes, appeal queue, escalation, later automation (P2)
Community Socket.IO Dart client risk	Pin version, integration tests; raw WebSocket fallback

7. Business Rules

ID	Rule
BR-01	Only verified university identities can access private institutional content.
BR-02	Only verified organizations or the Career Office can post job and internship listings; each listing expires after 14 days.
BR-03	Every moderation sanction needs a reason code and is logged immutably; sanctioned users can appeal once per action.
BR-04	A moderator or owner cannot review an appeal against their own decision.
BR-05	Messages from non-connections go to Message Requests; media is hidden until accepted.
BR-06	Project workspaces are visible only to the approved roster.
BR-07	Email domain verification is not proof of current enrollment; verification expires after 1 year and must be renewed.
BR-08	Only status, method, institution and dates are stored for verification; ID documents are never stored.

8. Use Case Specifications

UC-01 Register and Verify University Email

Primary actor	Student
Preconditions	App installed; user has a university email.
Main flow	1. User enters email and password. 2. System validates domain, creates a pending account and sends OTP. 3. User enters OTP. 4. System sets status VERIFIED and enrolls the user in the institutional community. 5. User completes onboarding.
Alternative flows	A1: Domain not allowed - show error. A2: OTP expired or wrong - allow retry/resend (max 5 attempts).
Postconditions	Verified account exists; institutional community joined; starter feed shown.

UC-02 Join Community

Primary actor	Student
Preconditions	User is verified and logged in.
Main flow	1. User opens the community preview. 2. System shows rules, member count and join mode. 3. User taps Join. 4. System adds the user to the roster.
Alternative flows	A1: Request Access - user answers up to 3 screening questions; moderator approves. A2: Invite-Only - join disabled.
Postconditions	User is a member (or pending request created).

UC-03 Post Discussion with Threaded Replies

Primary actor	Member
Preconditions	User is a member with posting rights.
Main flow	1. User opens the composer and selects Discussion. 2. User enters title, body, optional code/image. 3. System validates and publishes. 4. Members reply and vote; the feed sorts by New or Best.
Alternative flows	A1: Community requires post approval - post enters review queue. A2: Offline - post is stored in the outbox and sent later.
Postconditions	Post visible in the community and in followers' feeds.

UC-04 Send Direct Message / Message Request

Primary actor	Student
Preconditions	Sender and recipient are verified.
Main flow	1. Sender writes a message. 2. App stores it in the outbox and shows it as pending. 3. Server persists and acknowledges. 4. If the sender is a non-connection, it arrives in the recipient's Message Requests. 5. Recipient accepts.
Alternative flows	A1: Recipient blocks - future messages rejected. A2: Connection lost - client retries with the same idempotency key.
Postconditions	Message stored once with a sequence number; delivery state updated.

UC-05 Publish Verified Opportunity

Primary actor	Verified Organization
Preconditions	Organization domain verified.
Main flow	1. Organization fills the listing form. 2. System validates the schema. 3. Listing goes to the review queue. 4. Reviewer approves. 5. Listing is published with a 14-day expiry. 6. Students apply.
Alternative flows	A1: Rejected - feedback returned, organization edits and resubmits.
Postconditions	Listing active and auto-closes at expiry.

UC-06 Moderate and Appeal

Primary actor	Community Moderator
Preconditions	A report exists in the queue.
Main flow	1. Moderator claims the report. 2. Selects a reason code and sanction. 3. System logs the action and notifies the author with an Appeal link. 4. Author appeals. 5. Secondary reviewer decides.
Alternative flows	A1: Severe violation - escalate to Platform Safety and suspend the account. A2: Report invalid - dismiss.
Postconditions	Immutable audit entries for action and appeal outcome.

9. System Models (UML Diagrams and Workflows)

The following models describe the system from several views: functional (use case, DFD), structural (class, ER, deployment), and behavioral (sequence, activity, state). All diagrams follow UML 2.5 conventions.

9.1 Use Case Diagram

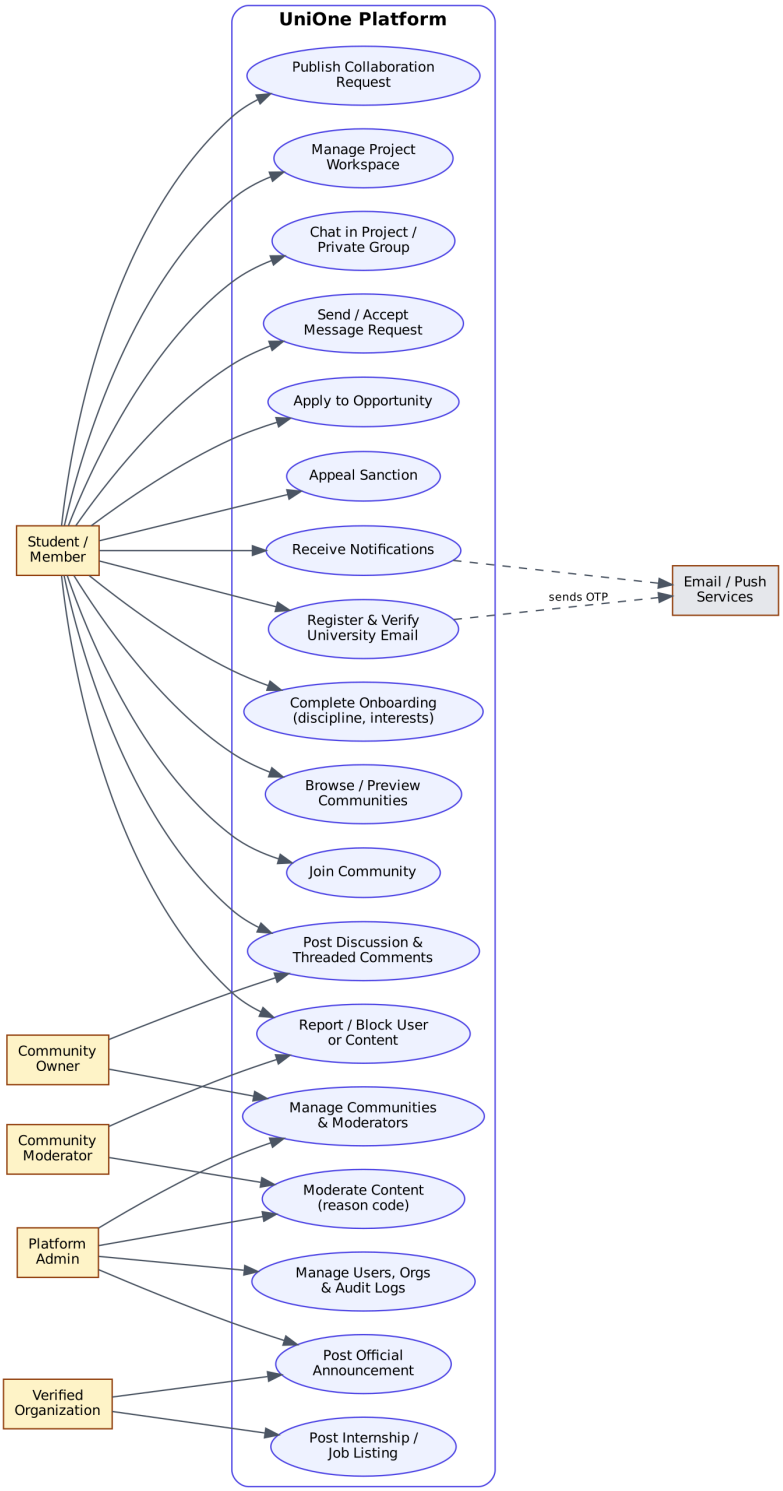


Figure 9.1 - Use case diagram of UniOne

9.2 Data Flow Diagram (Level 0)

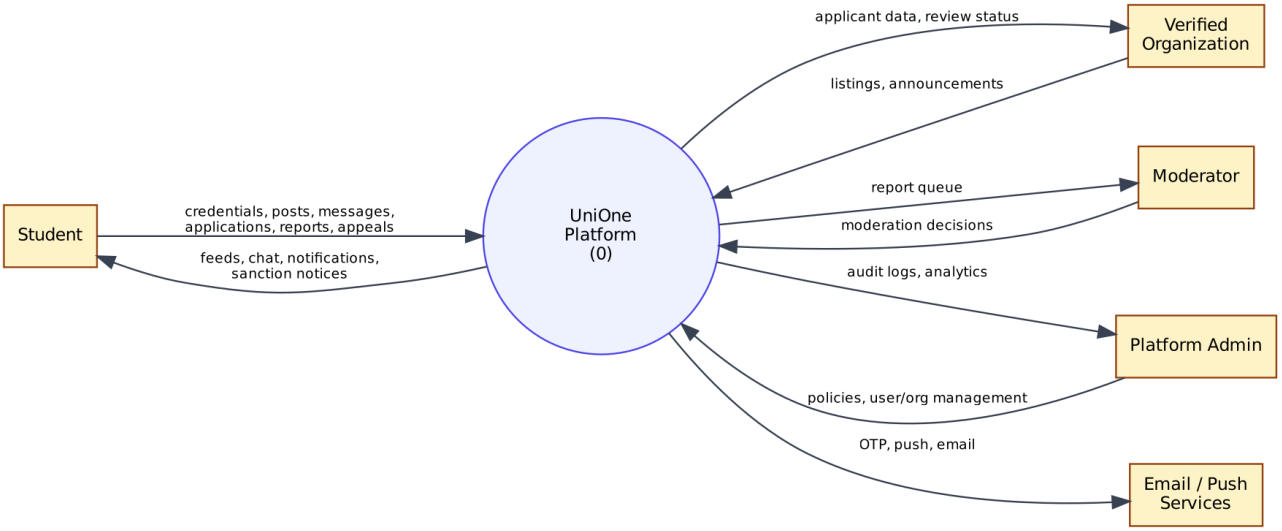


Figure 9.2 - Context-level data flow diagram

9.3 Class Diagram

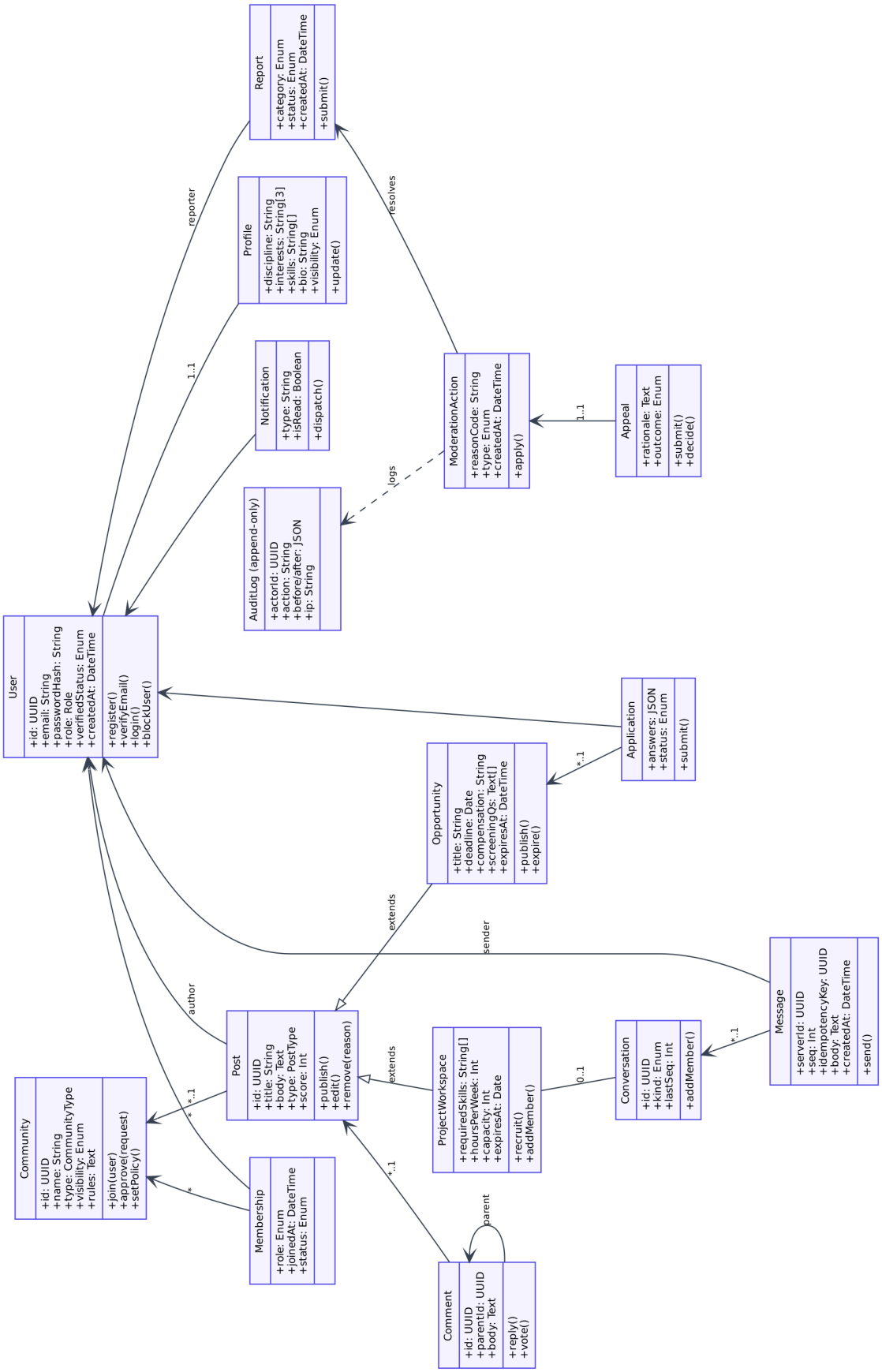


Figure 9.3 - Domain class diagram (rotated 90 degrees; core classes)

9.4 Entity Relationship Diagram

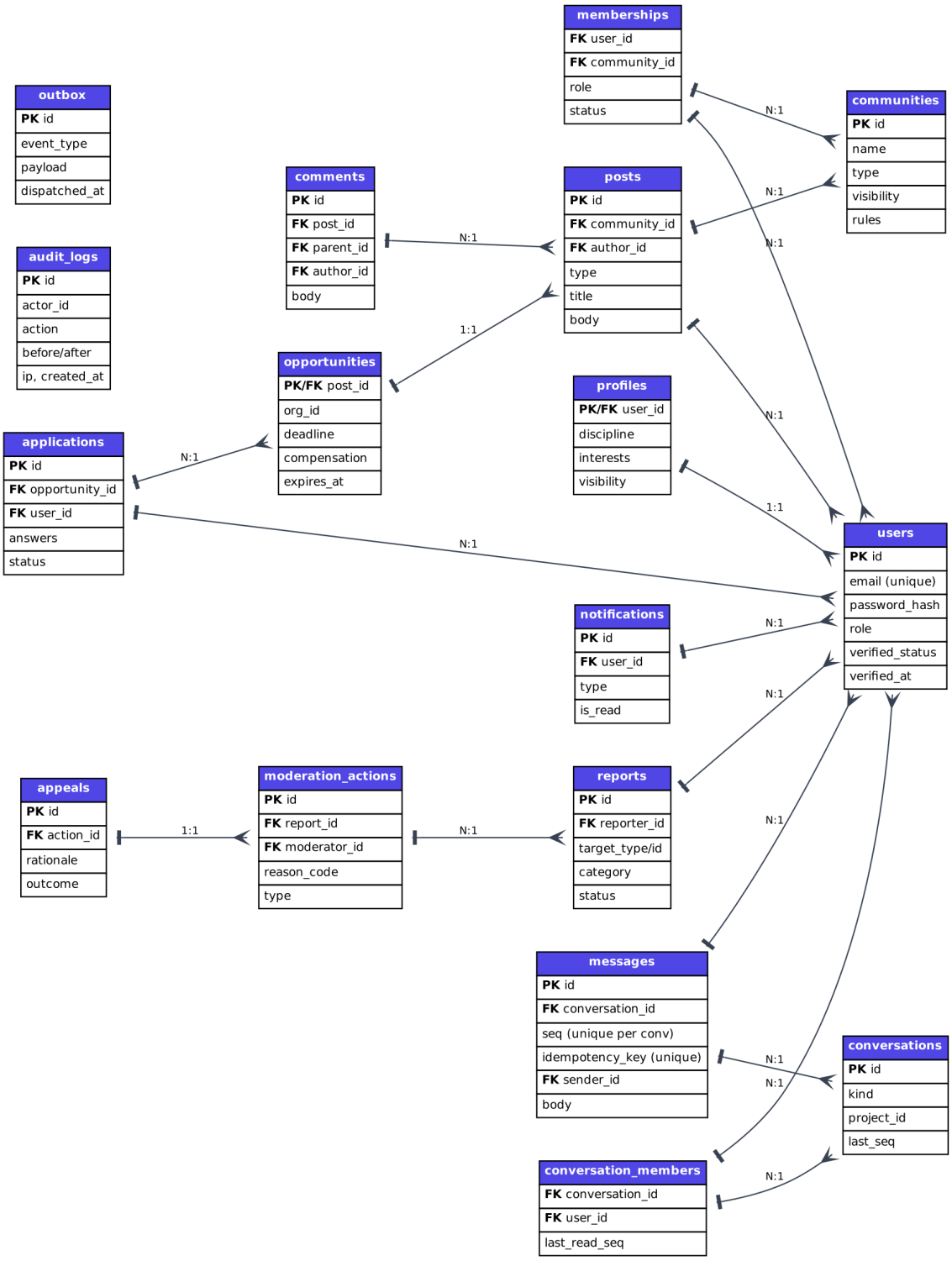


Figure 9.4 - Database ER diagram (PostgreSQL)

9.5 Sequence Diagrams

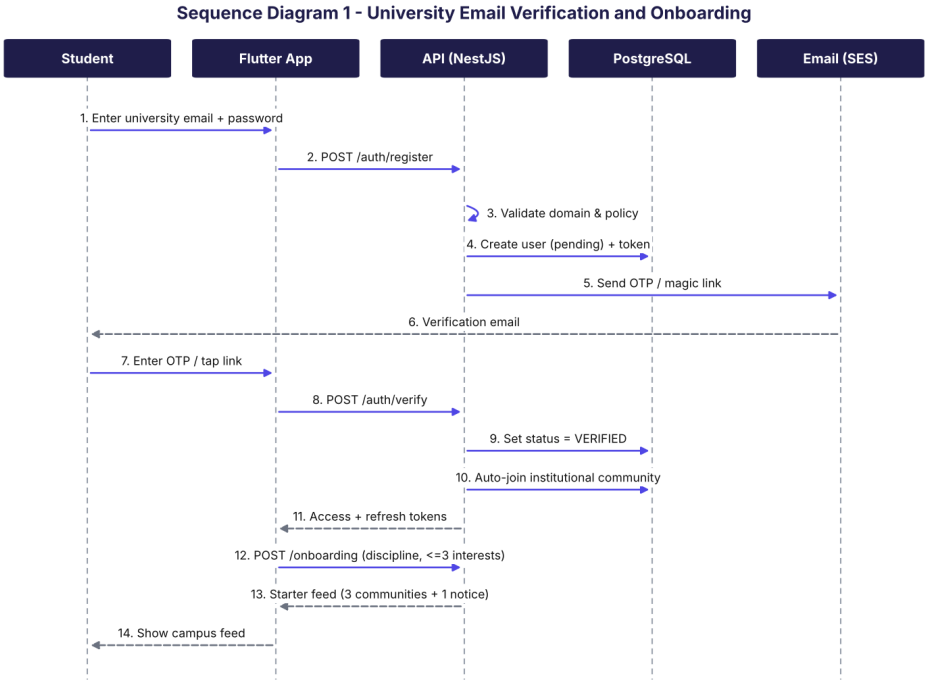


Figure 9.5 - Registration, verification and onboarding

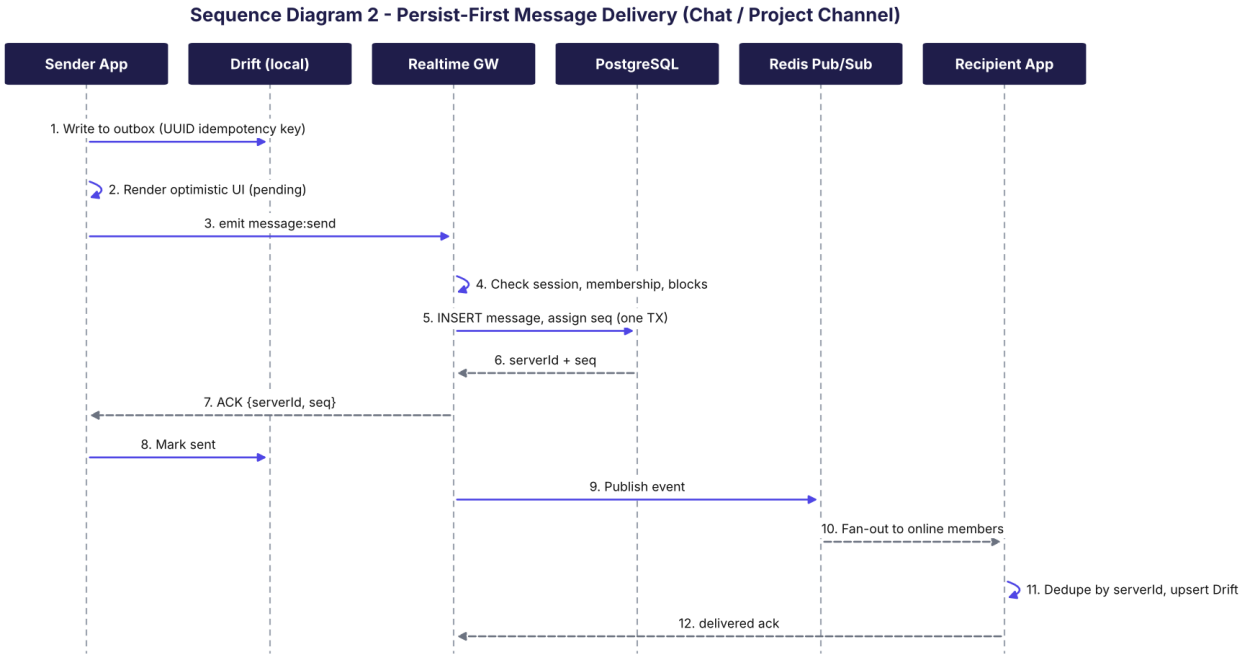


Figure 9.6 - Persist-first message delivery

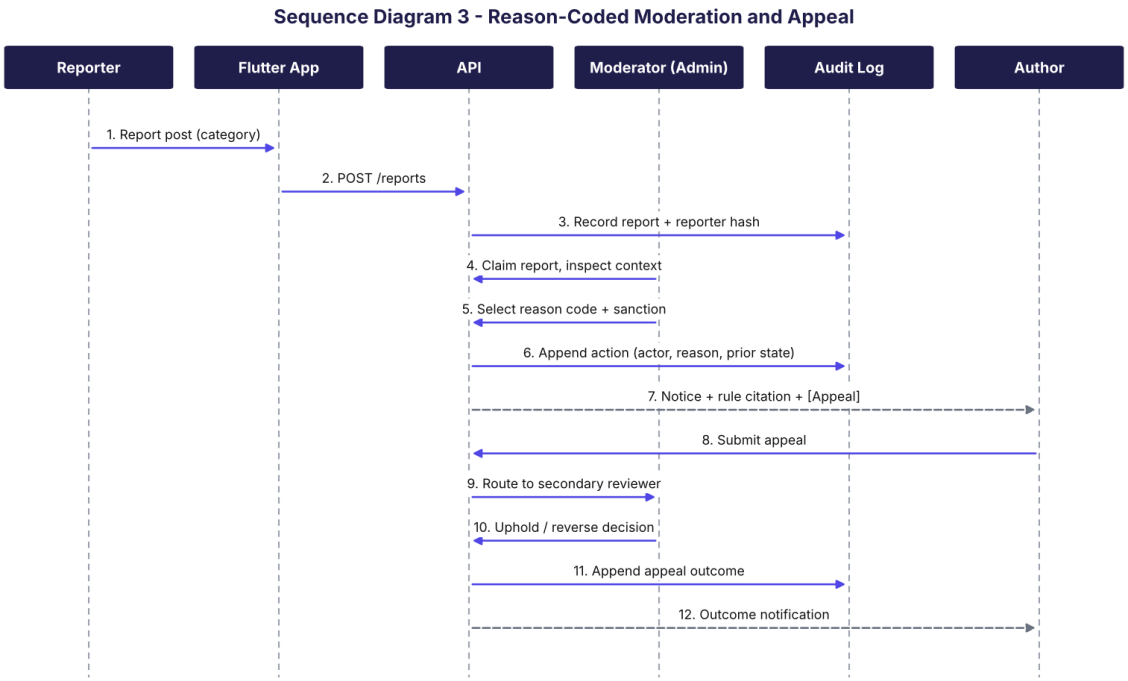


Figure 9.7 - Moderation action and appeal

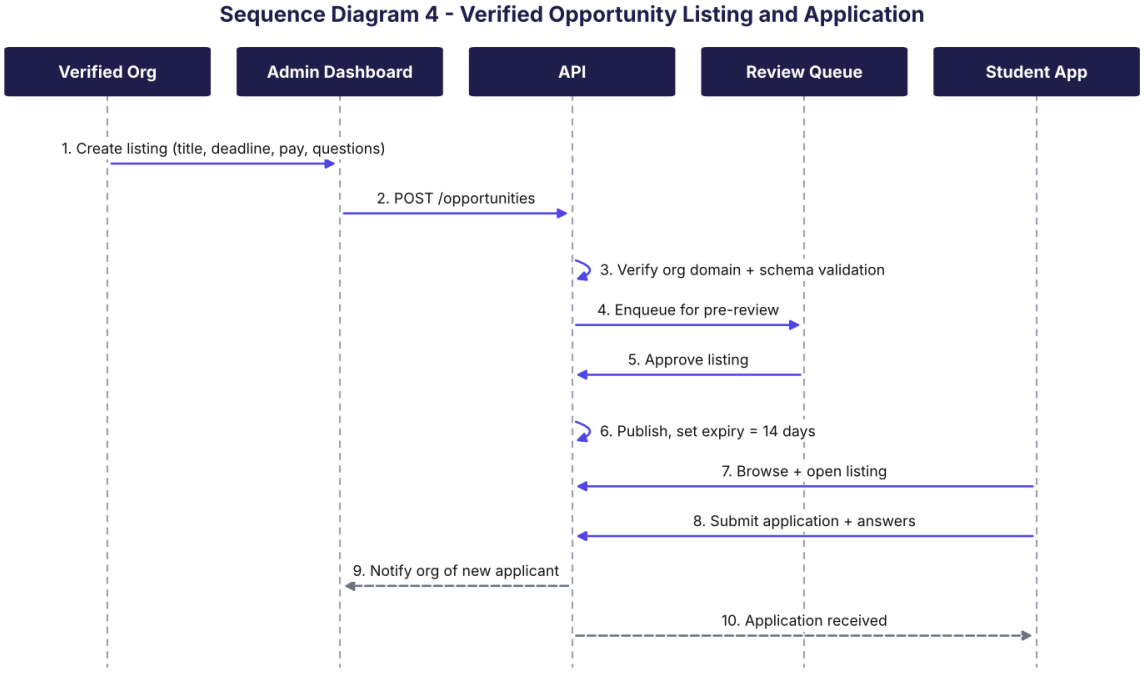


Figure 9.8 - Opportunity listing review and application

9.6 Activity Diagrams (Workflows)

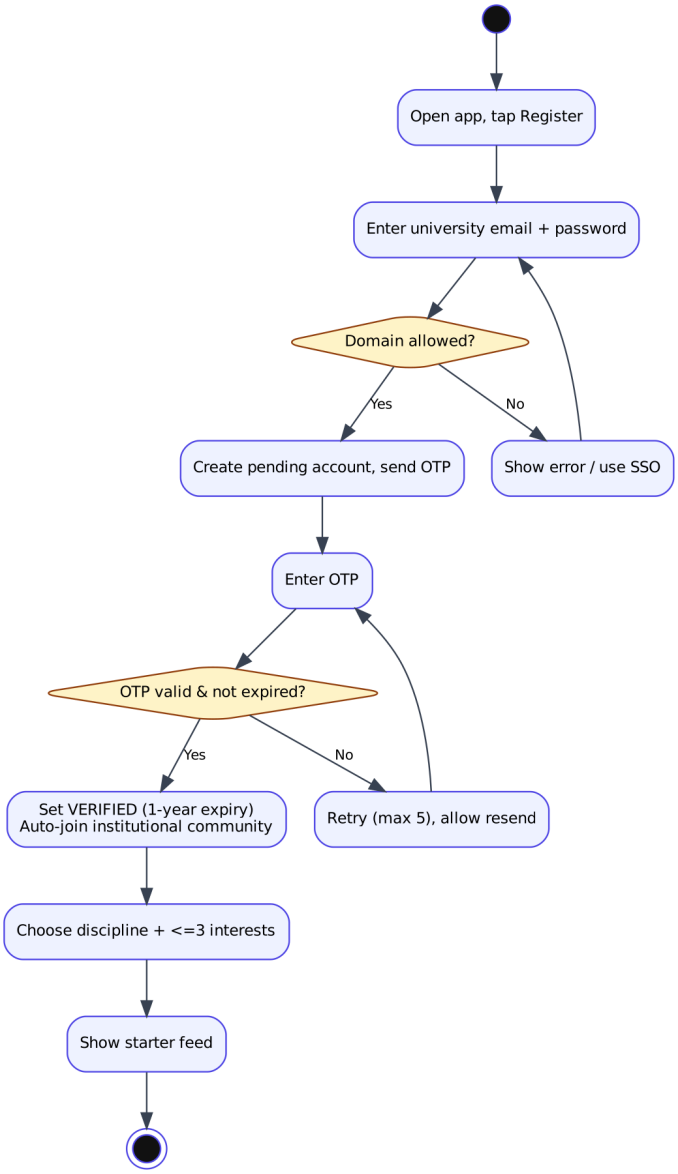


Figure 9.9 - Registration and verification workflow

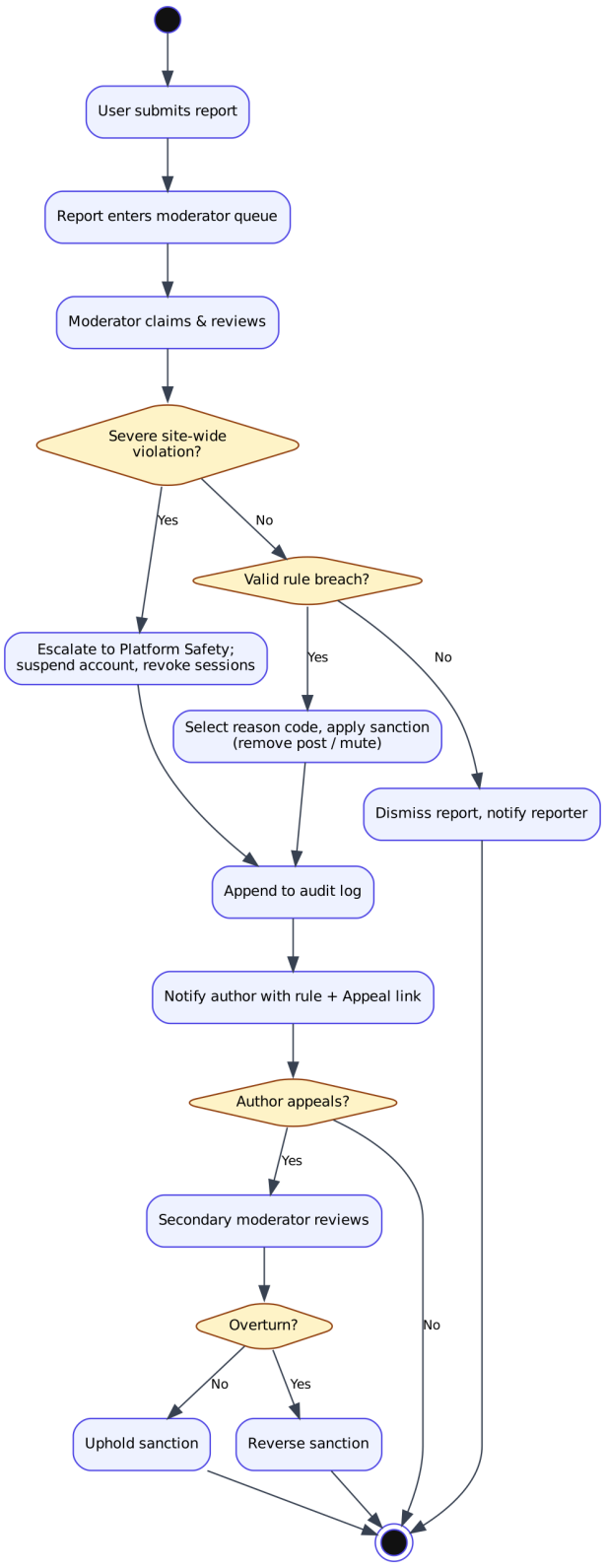


Figure 9.10 - Moderation and appeal workflow

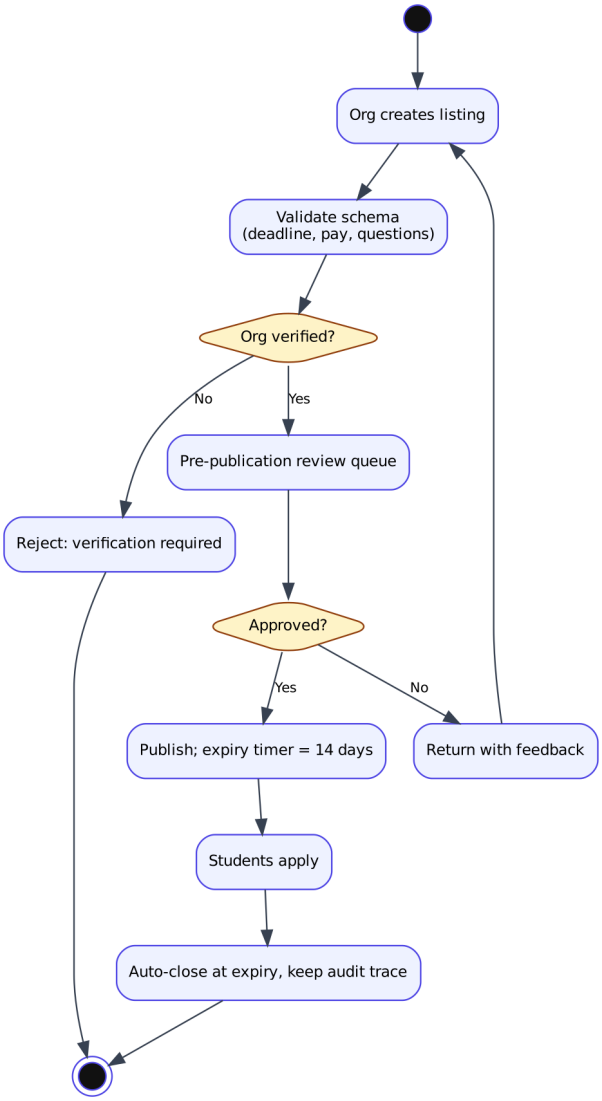


Figure 9.11 - Opportunity listing workflow

9.7 State Machine Diagrams

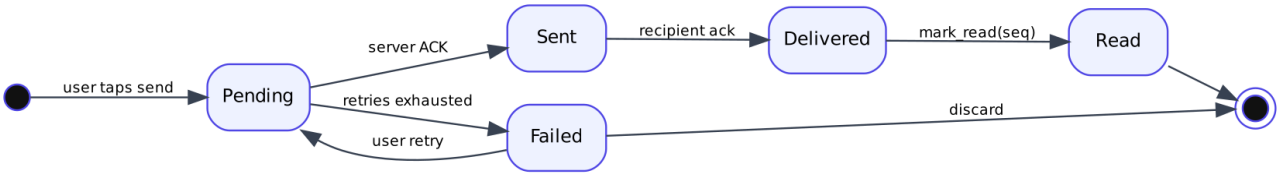


Figure 9.12 - Message lifecycle

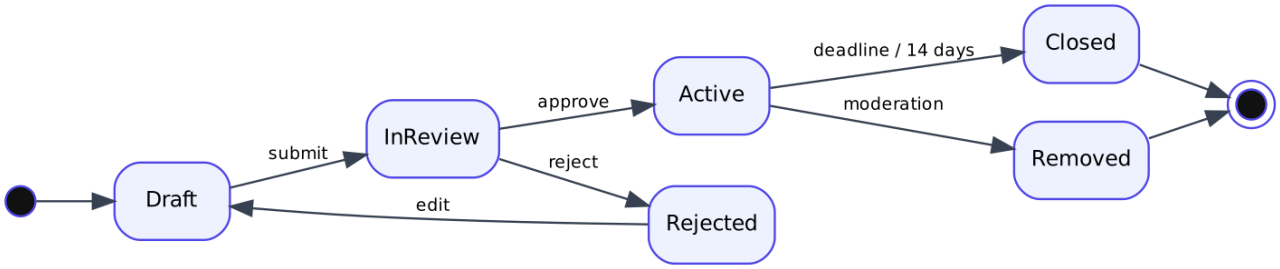


Figure 9.13 - Opportunity listing lifecycle

9.8 Deployment Diagram

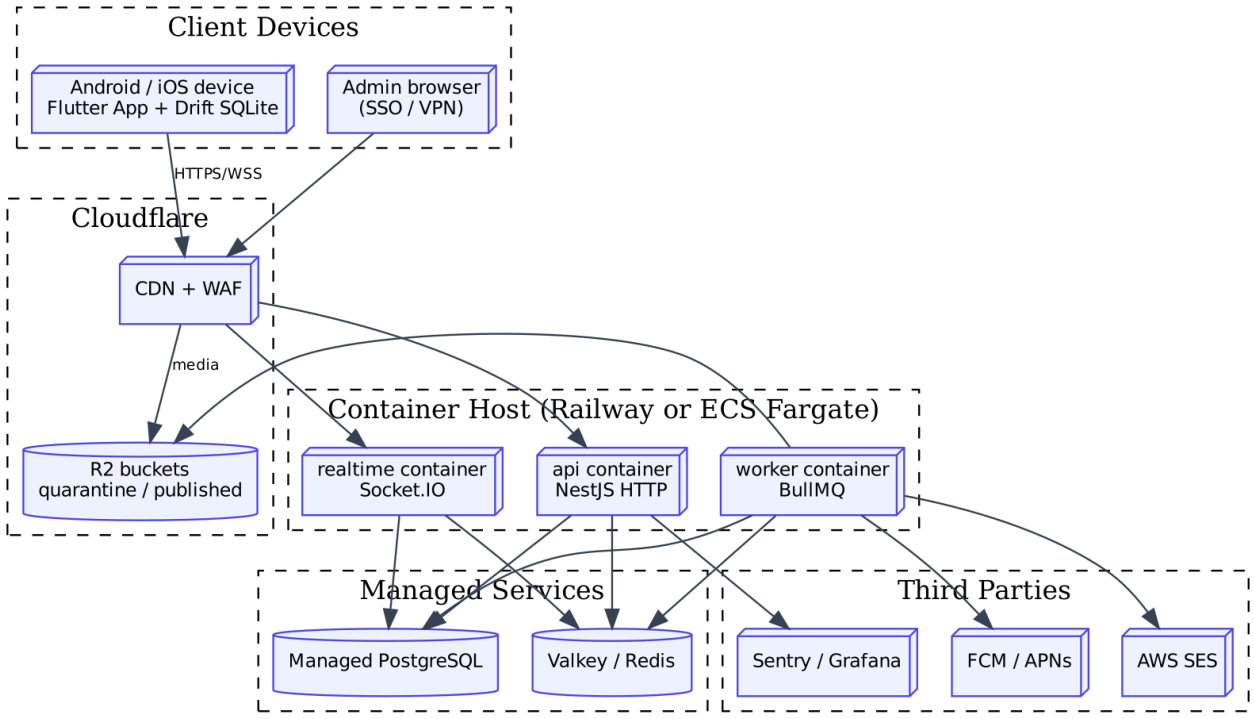


Figure 9.14 - Deployment view

10. Delivery Plan and Appendices

10.1 MVP Backlog Priorities

Pri.	Epic	Description
P0	Identity and verification	University email verification, profile, verified badge
P0	Role-based authorization	Server-side guards and anti-IDOR test suite
P0	Core community types 1-7	Create, configure visibility, join CTAs
P0	Threaded feed and sorting	Posts, comments, New/Best, report flags
P0	Verified opportunities	Structured listings, verified gate, 14-day expiry
P0	Safety and moderation	Block/report, message requests, reason codes, appeals, logs
P1	Collaboration requests, starter feed, project workspaces	Teammate recruitment, cold-start fix, private team space
P2	Moderation automation, industry and mentorship, voice/video	Phase 2/3 after identity matures

10.2 Thin Vertical Slice and 14-Day Plan

First milestone: Verified Student, Join Open Community, Publish Discussion, Report Post, Moderator reason-coded action, Author notice and appeal.

Days	Deliverable
1-2	Scope sign-off; OpenAPI 3.1 contract; generated TS and Dart types
3-5	PostgreSQL schema (Prisma), NestJS auth module, seed communities
6-8	Flutter core: Riverpod graph, Drift store, outbox pipeline, onboarding screens
9-10	Socket.IO gateway, discussion feed with New/Best sorting
11-12	Admin dashboard: moderation queue, reason-coded removal, appeal flow
13-14	Anti-IDOR tests, deployment, pilot cohort rollout

10.3 Pre-Scale Validation Gates

Question	Gate
Will students complete verification?	Completion at least 70%; onboarding under 3 minutes
Will employers post real internships?	Employer verification at least 80%; at least 3 genuine listings in week 1
Do students accept visible profiles?	Privacy opt-in at least 60%
Can students sustain moderation?	Queue turnaround under 24 h
Which join gate works best?	Request abandonment under 30%; safety rating at least 4.2/5

10.4 Requirements Traceability (User Story to Requirements)

User story	Description	Requirements
US01	Verify university email	FR-01 to FR-04
US02	Onboarding discipline and interests	FR-07, FR-08
US03	Preview community before joining	FR-11, FR-12
US04	Discussion with threaded replies	FR-15 to FR-18
US05	Structured collaboration request	FR-19, FR-20
US06	Verified org posts internship	FR-29, FR-30
US07	Message request queue	FR-26, FR-28
US08	Reason-coded removal	FR-35, FR-36
US09	Appeal a sanction	FR-37, FR-39
US10	Private project workspace	FR-21, FR-22

10.5 Technology Stack Summary

Layer	Choice
Mobile	Flutter, Riverpod 3, go_router, Drift, Dio, Freezed
Backend	Node.js 24 LTS, NestJS modular monolith (api + realtime + worker)
API	REST with OpenAPI 3.1; Socket.IO for real-time
Database	PostgreSQL with Prisma 7 and reviewed raw SQL
Cache / jobs	Valkey/Redis, BullMQ, transactional outbox
Media	Cloudflare R2 with CDN and presigned uploads
Push / email	FCM and APNs; AWS SES
Admin	Vite, React, TypeScript, TanStack Query/Table
DevOps	Monorepo, GitHub Actions, Docker, Railway or ECS Fargate, Terraform

End of document.